MS2 Summary — Padel Predictor

Noah Rodriguez · github.com/Noah-Rod/padel-predictor

1 What I achieved in this milestone

What works now that did not before. At MS1 the system was a design. Now the feature pipeline runs live every day against padelapi.org, and the training pipeline already runs on what it stores. By MS2 deliverable:

- **Automated ingestion from a live source, with backfill.** `HttpMatchSource` (`features/sources.py`) reads padelapi.org as its OpenAPI spec describes: tournaments at the proposal's levels (Premier Padel, FIP Platinum and Gold), then each one's main draw, paged and paced under the plan's limit (55 requests/min on Plus). Walkovers, byes, undrawn pairs and hidden free-plan results are dropped, not turned into fake fixtures; rankings are one weekly snapshot of official points and provider Elo. `padel backfill --start --end` loads history in upserted 30-day chunks, so an interrupted run resumes; loaded: February 2025 to November 2026, 2,076 requests.
- **Feature engineering and storage.** Six team-difference features (Elo, ranking points, form, rest, partnership, tier), replayed in match order by `common/features.py`, the one definition training and inference both import. Stored in Hopsworks as offline feature groups with a primary key and an event time: `raw_matches`, `raw_rankings`, `match_features` and `predictions`, all upserted.
- **Scheduled, not manual.** GitHub Actions runs the feature pipeline daily at 04:15 UTC (seven days back, seven ahead), training after every successful feature run and on Mondays, and inference daily at 06:00. Each feature run ends with `padel data-report` in its step summary. Keys live in GitHub Secrets and a git-ignored `.env`, never in the repository.
- **Docker image, pinned dependencies, informative README.** `Dockerfile` and `docker-compose.yml`, `uv.lock`, 124 unit tests and lint in CI on Python 3.11 and 3.12. The README has clone-and-run steps, a diagram of tools and triggers (`docs/system.svg`) and the measured API and Actions limits; without a key, a sample source runs everything offline.

Key decisions, and why.

- *Rebuild every feature on every run* instead of appending rows: Elo and form depend on match order, so a late result would silently shift every later row.
- *Join rankings on their publication time*, strictly before the match (E1): a snapshot labelled with a tournament week can already contain that week's results.
- *Hopsworks and W&B, as in the proposal, not local Parquet and MLflow* (my first version): a GitHub runner starts with an empty disk, so local stores lost everything between runs. Offline backends with the same contract keep the tests free of accounts.
- *Keep the free-plan and paid-plan results side by side*, tagged per plan in W&B: the free plan is the fallback anyone can run (E2), the paid plan is the real test.

Look at first. Three places, in this order: (1) the latest *Feature pipeline* run in the repository's Actions tab: its step summary ends with the data report, the real counts and baselines read back from Hopsworks; (2) `attach_rankings` in `common/features.py` and the leakage tests at the end of `tests/test_features.py`: the publication-time join the editors asked for; (3) `parse_records` in `features/sources.py`: the only code that knows padelapi.org's payload, and what it drops.

What it cost. *Data:* padelapi.org has about 2,000 main-draw matches a season, not the 3,000 to 4,000 I estimated, and I backfilled from February 2025 instead of 2023, so the test set is 793 matches instead of about 2,000, and only 63 % of them have a provider rating. *Accuracy:* on those rated matches the model does not beat the provider yet (table). *Rewrites:* storage moved from Parquet and MLflow back to Hopsworks and W&B; the live adapter was rewritten against the real API; and the feature groups were recreated as DELTA (v2), because Hopsworks' default Kafka upload needs a keystore the Python client cannot read. Going live also needed fixes the stand-in tests could not catch: `tee` hid failed runs (`a3ecd9d`), same-day rounds replayed out of order leaked a quarter-final into its round of 16 (`4827680`), and a failed Hopsworks read passed for an empty table (`8b32bcf`).

Results, free and paid plan. Without ranking history the free plan's provider bar is a constant 0.5; on the Plus plan, the headline gap comes from the unrated 37 %.

	Free plan · run `1tq3lkm7`	Plus plan · run `j99rrfnw` (champion v1)
Labelled matches · test set	1,674 · 330 from 2026-10-12	3,995 (from 2025-02-12) · 793 from 2026-09-01
Model: log loss / accuracy	0.559 / 68.5 %	**0.510 / 77.2 %**
Provider Elo (coverage)	0.693 / 54.8 % (0 %)	0.574 / 67.1 % (63 %)
In-house Elo · majority class	0.632 · 0.692	0.575 · 0.693
Model − provider, 95 % CI	not meaningful (bar is 0.5)	[−0.097, −0.031], flattered by the 0.5 fill
Rated matches only	–	500: model 0.523 vs provider **0.505**, CI **[−0.022, +0.060]: not met**

2 Response to MS1 reviews

Labels as in the editorial summary. Reviewer A's suggestion is A2, reviewer B's suggestion is B1. Code: commits `a2fb2a5`, `9a3913f` and `9efda66` (live source); documentation: `0b5feb8`.

- **A2 · Baseline is not measured on my test set — Changed** (also reviewer A's suggestion). The provider's Elo comes from the snapshot published strictly before each match, is stored as `provider_elo_prob` beside each feature row (never as a model input) and is scored on exactly the model's test rows: 0.574 on 793 matches, not the provider's 0.517 on its own 7,500. An unrated match gets 0.5 rather than being dropped, and training also compares on the rated matches alone (`covered_comparison`, `797522c`), where the provider is ahead: **the success criterion is not met yet** (table). MS3: read the full ranking lists (coverage is 63 % with the top 200), register the provider's Elo as the first champion, run the ablations (E3). *Where:* `training/dataset.py`, `features/report.py`.
- **E1 · Weekly snapshots can leak same-week results — Changed.** Rankings are their own raw dataset with a `published_at` column, and each player gets the latest snapshot published *strictly before* the scheduled start (`merge_asof` without exact matches), never the one matching the label date. A backfilled snapshot without a publication time counts as published seven days after its label date, so it is never visible inside its own tournament week. Tests: a snapshot published after the week, one published exactly at the start, and rewriting every later snapshot leaves the match's row unchanged. *Where:* `common/features.py` (`attach_rankings`), `common/schemas.py`, `tests/test_features.py`.
- **A1 · No minimum margin on ~2,000 test matches — Changed.** A paired bootstrap (2,000 resamples) of the per-match log-loss difference, model minus provider Elo. The criterion is met only if the whole 95 % interval lies below zero; it is logged as `vs_provider_elo_ci_low/high` and the tag `beats_provider_elo`. An interval rather than a fixed margin, because a margin would have been a guess about a test-set size I did not know yet. A test checks that a 0.001 edge on 2,000 matches does not count as a win. *Where:* `training/dataset.py` (`bootstrap_log_loss_difference`), `config/settings.yaml`.
- **E2 · Paid data tier: real volume and a fallback — Changed.** Real counts from the store (`padel data-report`, Plus plan): 3,995 labelled main-draw matches from February 2025 — 2025: P1 708, P2 412, Major 335, FIP Platinum 232, FIP Gold 60, Finals 14; 2026: P1 663, FIP Gold 649, P2 448, Major 258, FIP Platinum 216. About 2,000 a season (estimate: 3,000 to 4,000); earlier history is a backfill away. The free fallback ran first (1,674 matches, 188 requests). `PADEL_API_PLAN=free` clamps the backfill to 183 days and stores only the current ranking, published at fetch time, so no historical match gets points it could not have had. *Where:* `features/report.py`, `features/sources.py`, README "Limits and quotas".
- **B1 · How player values become team features — Changed.** Elo: mean of the two players. Ranking points: sum, from the snapshot. Form: mean of each player's win rate over their last 5 matches with any partner, 0.5 without history. Rest: the smaller of the two players' gaps, capped at 60 days, 14 if never seen. Partnership: matches this exact pair has played together. A new partnership starts at 0 but keeps each player's Elo, form and rest; a test covers this. *Where:* README "What the model sees", `common/features.py` docstring, `tests/test_features.py`.
- **E3 · Report what partnership and rest add — Not yet.** Ablations are training work (MS3): partnership, form and rest are each dropped in turn and every run's interval against the provider's Elo compared in W&B. Until then the feature set stays at six, and the provider's Elo is kept out of it as a clean bar. *Where:* `training/pipeline.py` (MS3).
- **E4 · Log which model served which day — Changed.** Every prediction row in `predictions` (key `match_id`, `run_date`) carries `model_name`, `model_version`, `model_run_id` and `predicted_at`, so the last pre-match prediction is the latest `predicted_at` before `played_at`. Scoring it live is MS4. *Where:* `inference/predict.py` (`predict_frame`), `tests/test_inference.py`.